JOBCONNECT USING JUNIT 5 AND MOCKITO

UNIT TESTS FOR BACKEND SERVICES.

Testing & Quality Assurance Report

Project: JobConnect Full-Stack Application **Tech Stack:** Spring Boot 3, React (Vite), MySQL, Spring Security (JWT), Twilio API

1. Backend Unit Testing (JUnit 5 & Mockito)

To ensure backend reliability and isolate business logic, unit tests were implemented across the three primary architectural layers using **JUnit 5** and **Mockito**.

A. Repository Layer Testing

- **Objective:** Validate custom database queries and data persistence.
- **Implementation:** Utilized the @DataJpaTest annotation to spin up an isolated, in-memory **H2 Database** specifically for testing, preventing any accidental modification of the production MySQL database.
- **Test Case:** Verified the findByTitleContainingIgnoreCaseOrLocationContainingIgnoreCase custom query in the JobRepository. Mock data was saved to the H2 database, and the test successfully asserted that the search method returned the correctly filtered results.

B. Service Layer Testing

- **Objective:** Validate core business rules and logic without relying on the database.
- **Implementation:** Utilized @ExtendWith(MockitoExtension.class). The actual database repositories were mocked using @Mock, and these fake repositories were injected into the ApplicationService using @InjectMocks.

- **Test Case:** Tested the rule preventing duplicate job applications. Mockito was instructed to simulate an existing application. The test successfully asserted that the service intercepted the request, threw the custom RuntimeException ("You have already applied for this job"), and verified via verify(repository, never()).save(...) that the database was not accessed.

C. Controller Layer Testing

- **Objective:** Validate HTTP routing, JSON serialization/deserialization, and Spring Security Role-Based Access Control (RBAC).
- **Implementation:** Utilized @WebMvcTest to load only the web layer for fast execution. Standard services and JWT utilities were bypassed using @MockBean.
- **Test Case:** Tested the secured POST /api/jobs/create endpoint using MockMvc and the spring-security-test library's @WithMockUser annotation.
    - **Authorized Case:** Simulated an EMPLOYER role request. Asserted an HTTP 201 Created response.
    - **Unauthorized Case:** Simulated a JOB_SEEKER role request. Asserted an HTTP 403 Forbidden response, proving the JWT role-based security successfully protected the endpoint.

2. End-to-End (E2E) Testing

Comprehensive E2E testing was manually executed to validate the full data lifecycle across the React frontend, the Spring Boot API, the MySQL database, and the external Twilio API.

Verified User Workflows:

Workflow 1: Authentication & Routing

- **Action:** A user registers and logs in via the React interface.
- **Validation:** The backend securely hashes the password, creates the user in MySQL, and returns a JWT containing the user's role. The React AuthContext successfully decodes this token and restricts frontend routing, pushing Employers to the Employer Dashboard and Seekers to the Job Search page.

Workflow 2: The Core Application Lifecycle

- **Action:** An Employer creates a job posting. A Job Seeker searches for the job and applies.
- **Validation:** Data flows seamlessly from the React forms via Axios to the Spring Boot controllers, and persists in the MySQL relational tables. The UI accurately reflects the database state (e.g., preventing duplicate applications via UI error messages).

Workflow 3: Third-Party API Trigger

- **Action:** The Employer updates an application status from "PENDING" to "HIRED" via the dashboard.
- **Validation:** The frontend triggers the backend PUT request. Spring Boot updates the database and successfully initializes the SmsService. The Twilio API connects and sends a real-time SMS text message to the Job Seeker's verified mobile device, completing the full-stack workflow.

Finished after 4.166 seconds

Runs: 1/1 Errors: 0 Failures: 0

ApplicationServiceTest [Runner: JUnit 5] (3.559 s)

Finished after 12.394 seconds

Runs: 1/1 Errors: 0 Failures: 0

JobRepositoryTest [Runner: JUnit 5] (1.135 s)

testSearchJobsByKeyword() (1.135 s)

Finished after 11.58 seconds

Runs: 2/2 Errors: 0 Failures: 0

JobControllerTest [Runner: JUnit 5] (1.199 s)

testCreateJob_AsEmployer_ReturnsCreated() (1.160

testCreateJob_AsJobSeeker_ReturnsForbidden() (0.

Finished after 21.206 seconds

Runs: 1/1 Errors: 0 Failures: 0

JobconnectApplicationTests [Runner: JUnit 5] (1.201 s

contextLoads() (1.201 s)

Package Explorer JUnit

Finished after 28.95 seconds

Runs: 5/5 Errors: 0 Failures: 0

JobconnectApplicationTests [Runner: JUnit 5] (1.354

contextLoads() (1.354 s)

JobRepositoryTest [Runner: JUnit 5] (0.098 s)

testSearchJobsByKeyword() (0.098 s)

JobControllerTest [Runner: JUnit 5] (1.427 s)

testCreateJob_AsEmployer_ReturnsCreated() (1.3

testCreateJob_AsJobSeeker_ReturnsForbidden()

ApplicationServiceTest [Runner: JUnit 5] (1.207 s)

testApplyForJob_AlreadyApplied_ThrowsExcepti